Experiment No 1

Documentation on Case Study: System Programming and Compilers

---

1. Introduction

Overview of System Programming

System programming involves the creation of software that manages and operates computer hardware, providing a foundation for application software to run efficiently. It includes tasks such as creating operating systems, device drivers, and utilities. Unlike application programming, which focuses on user-level software, system programming interacts directly with hardware and system resources.

Purpose of the Case Study

This case study focuses on system programming with an in-depth exploration of compilers, one of the most crucial components in system software development. A compiler is a tool that translates high-level programming languages into assembly code or machine code, enabling a computer to understand and execute the instructions written by a developer. The goal of this case study is to analyze the architecture and functioning of compilers, providing a deeper understanding of how they optimize and generate assembly code from high-level programming languages.

---

2. Background Information

System Programming Basics

System programming encompasses a range of low-level programming tasks, such as working with assembly language, managing memory, and interacting with system calls. The primary types of system programs include:

- **Assemblers**: Programs that convert assembly language into machine code.
- **Linkers**: Programs that combine object files into executable files.
- **Loaders**: Programs that load executable files into memory for execution.
- **Interpreters**: Programs that execute instructions directly without compiling them into machine code.

These system programs ensure the smooth functioning of the operating system and provide the necessary infrastructure for application programs.

What is a Compiler?

A compiler is a special type of system program that translates source code written in a high-level language into machine code or an intermediate form(assembly code), which can then be executed by a

computer. A compiler performs several tasks, including lexical analysis, parsing, semantic analysis, optimization, and code generation. The end result is an assembly code file that will go to assembler.

The process of compiling code typically involves multiple stages:

1. **Lexical Analysis**: Breaking down the source code into tokens.
2. **Syntax Analysis**: Analyzing the structure of the source code.
3. **Semantic Analysis**: Ensuring the code is meaningful and error-free.
4. **Optimization**: Improving code efficiency and performance.
5. **Code Generation**: Converting the code into assembly language.

---

3. Case Study: Compiler

Objective

The primary objective of this case study is to explore the design and implementation of a compiler, focusing on the different phases of compilation, the optimization techniques used, and the challenges faced in creating a functional compiler.

Approach

The approach to this case study involved both research and hands-on experimentation. I researched various compilers to understand their inner workings and followed the implementation of a simple compiler from scratch. The case study specifically examines the design of a compiler for a custom programming language that includes basic arithmetic operations, variables, and control flow.

Methodology

The methodology for this case study included:

- **Literature Review**: Understanding the theoretical aspects of compiler design from books and research papers.
- **Hands-on Development**: Building a basic compiler using Lex and Yacc (tools for lexical analysis and syntax parsing).
- **Testing and Evaluation**: Evaluating the compiler by running sample programs and analyzing its efficiency and output.

---

4. In-depth Exploration of the Compiler

Compiler Design and Architecture

A compiler typically has three main components:

- **Front-end**: The front-end processes the source code and performs lexical and syntax analysis. It generates an intermediate representation (IR) of the code.

- **Middle-end**: This phase performs optimizations on the IR, making the code more efficient without changing its behavior.
- **Back-end**: The back-end translates the optimized IR into machine code or assembly language, which can be executed by the system.

Detailed Explanation of Compiler Phases

1. **Lexical Analysis**:
   - The compiler reads the source code character by character, grouping them into tokens, such as keywords, operators, and identifiers. The output is a list of tokens that represent the syntax of the program.
   - Example: For the code `x = 5 + 3;`, the lexer will generate tokens for `x`, `=`, `5`, `+`, `3`, and `;`.
2. **Syntax Analysis**:
   - The syntax analyzer, or parser, checks whether the sequence of tokens forms valid statements according to the language's grammar. It creates a **parse tree** or **abstract syntax tree (AST)** that represents the structure of the program.
   - Example: The parser might transform the expression `x = 5 + 3` into a tree that shows the assignment operation as the root, with the operands as its children.
3. **Semantic Analysis**:
   - This phase checks for logical errors and ensures the program makes sense. For example, it ensures that variables are declared before use and that type consistency is maintained.
   - Example: If a programmer tries to add a string to an integer, the semantic analyzer will detect the type mismatch.
4. **Optimization**:
   - In this phase, the compiler attempts to improve the performance and efficiency of the code. This could include eliminating redundant calculations, simplifying expressions, or using more efficient data structures.
   - Example: If the code has an expression `x + 0`, the compiler might optimize this to just `x`.
5. **Code Generation**:
   - The final phase translates the intermediate code into machine code or assembly code that can be executed by the target machine. This code is often specific to the architecture of the processor.
   - Example: The expression `x = 5 + 3` might be converted into machine instructions for adding `5` and `3` and storing the result in the memory location of `x`.

Challenges Encountered

- **Handling Ambiguities in Language Syntax**: One of the challenges was ensuring that the language grammar was well-defined to avoid ambiguity in parsing.
- **Optimization Trade-offs**: Optimizing code for performance without making it too complex or hard to debug was a significant challenge. It required balancing efficiency with maintainability.
- **Debugging Compiler Errors**: Debugging errors in the compiler itself proved to be difficult, especially when dealing with complex interactions between different phases of the compiler.

---

5. Results and Findings

The case study revealed that while the core phases of a compiler can be implemented using tools like Lex and Yacc, optimization remains one of the most complex and nuanced parts of the compiler design. The custom language compiler developed as part of this study successfully compiled simple arithmetic expressions, but further work is needed to improve error handling and optimize code generation.

---

6. Conclusion

Compilers are a crucial component of system programming, enabling developers to write high-level code that can be executed on different platforms. Through this case study, it was found that while building a simple compiler is achievable with modern tools, challenges such as optimization and error handling remain significant. The study underscores the importance of compilers in the software development ecosystem and provides a foundation for further exploration into more advanced compiler design techniques.

---

7. References

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. 2nd Edition. Pearson.
- Appel, A. W. (2002). *Modern Compiler Implementation in C*. Cambridge University Press.
- Keshav, S. (2011). *An Engineering Approach to Computer Programming*. Pearson.

8. Appendices

Appendix A: Sample Code

```
int main() {
    int x = 5 + 3;
    printf("Result: %d", x);
    return 0;
}
```

Appendix B: Compiler Output

```
Compiling: main.c
Optimization: Simplified addition
Generated Code: MOV R1, 5
                ADD R1, 3
                MOV x, R1
                CALL printf
```

Experiment No 2

Q.1 Check the no is Even or Odd.

```
/*Program to check whether entered no is even or odd */
/*Declaration Section(header files,variable declaration if)*/
%{#include<stdio.h>
#include<stdlib.h>
%}
/*Rules section*/
%%
[+-]?[0-9]+ {
int i=atoi(yytext);
if(i%2==0){
printf("It is Even Number");
}
else{
printf("It is odd Number");}
}
.* {
printf("Invalid input");}

%%
int yywrap(){}              //main section
int main(){
yylex();
return 0;
}
```

Output:

```
C:\Users\comp\Desktop>flex Exp2A.l

C:\Users\comp\Desktop>gcc lex.yy.c -o Exp2A

C:\Users\comp\Desktop>Exp2A
1
It is odd Number
2
It is Even Number
-2
It is Even Number
@
Invalid input
a
Invalid input
```

Q.2 Program to check wheather entered character is Vowel or Consonent

```
/*Program to check wheather entered character is Vowel or Consonent*/
%{
#include<stdio.h>
int vowel=0;
int conso=0;
%}
%%
"a"|"e"|"i"|"o"|"u"|"A"|"E"|"I"|"O"|"U" {printf("Entered character is Vowel");vowel++;}
[a-zA-Z] {printf("Enter character is consonent");conso++;}
.* {printf("Invalid Input");}
%%
int yywrap(){}
int main(){
printf("Enter the characters:");
yylex();
return 0;}
```

Output:

```
C:\Users\comp\Desktop>flex Exp2B.l

C:\Users\comp\Desktop>gcc lex.yy.c -o Exp2B

C:\Users\comp\Desktop>Exp2B
Enter the characters:a
Entered character is Vowel
A
Entered character is Vowel
p
Enter character is consonent
P
Enter character is consonent
0
Invalid Input
$
Invalid Input
^Z
At the end Count of vowel is 2 and count of Consonent 2
```

Q.3 Program for counting no of characters or numbers.

```
%{
#include<stdio.h>
int i=0;
%}
%%
[a-zA-Z0-9] {i++;}
"\n" {printf("Count: %d\n",i);i=0;}
. {}
%%
int yywrap(){}
int main(){
yylex();
```

```
return 0;
}
```

Output:

```
C:\Users\comp\Desktop>flex Exp2C.l

C:\Users\comp\Desktop>gcc lex.yy.c -o Exp2C

C:\Users\comp\Desktop>Exp2C
{PRATIK}
Count: 6
PRATIK ZINJAD
Count: 12
123456789
Count: 9
!@#$%^&*()_+}{:"><12345
Count: 5
```

Q.4 To identify relational operators.

```
%{
#include <stdio.h>
#include <stdlib.h>
%}
%option noyywrap

%%
==   { printf("Encountered IsEqual to operator.\n"); }
!=   { printf("Encountered Not equal to operator"); }
>    { printf("Encounter Greater than operator"); }
>=   { printf("Encounter Greater than equal to operator"); }
\<   { printf("Encounter less than operator"); }
\<=  { printf("Encounter less than equal to operator"); }
.    { printf("Invalid input"); }
%%

int main() {
    printf("Enter input:\n");
    yylex();
    return 0;
}
```

```
C:\Users\comp\Desktop>flex Q4.l

C:\Users\comp\Desktop>gcc lex.yy.c -o Q4

C:\Users\comp\Desktop>Q4
Enter input:
==
Encountered IsEqual to operator.

<=
Encounter less than equal to operator
<
Encounter less than operator
>
Encounter Greater than operator
=
Invalid input
+
Invalid input
```

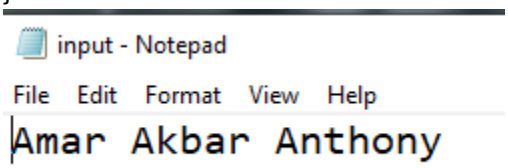
Q.5 To Copy the content from one file to another.

```
%{
#include <stdio.h>
#include <stdlib.h>
FILE *outFile;
%}
%option noyywrap

%%
. {fputc(yytext[0],outFile);}
%%

int main(int argc,char *argv[]) {
    if(argc!=3){
            printf("You should give lexfilename inputfile outputfile");
            return 0;
            }
            FILE *inFile=fopen(argv[1],"r");
            if(inFile==NULL){
            printf("There is error while opening file");
            return 0;
            }
            outFile=fopen(argv[2],"w");
            if(outFile==NULL){
            printf("There is error while opening file");
            return 0;
            }
            yyin=inFile;
    yylex();
            printf("File copied successfully.");
```

```
    return 0;
}
```



Experiment No 03

Aim: To Perform Arithmetic Operation using YACC.

Exp3.y

```
%{
#include<stdio.h>
extern int yylex();
extern void yyerror(const char *s);
%}
%token NUMBER
%left '+' '-'
%left '*' '/' '%'
%left '(' ')'

%%
ArithmeticExpression:
E {
printf("Result %d",$$);
return 0;};
E:E'+'E {$$=$1+$3;}
|E'-'E {$$=$1-$3;}
|E'*'E {$$=$1*$3;}
|E'/'E {$$=$1/$3;}
|E'%'E {$$=$1%$3;}
|'('E')' {$$=$2;}
|NUMBER {$$=$1;};
%%

int main(){
printf("Enter expression:");
return yyparse();

}
void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}
```

Exp3lex.l

```
%{
#include "Exp3.tab.h"  // Include the header generated by Yacc/Bison
%}

%%
[0-9]+          { yylval = atoi(yytext); return NUMBER; }  // Number token
[\+\-\*/%]      { return yytext[0]; }  // Operator tokens
[ \t]+        ;  // Skip whitespace
\n             {return '\n';}
.              { return yytext[0]; }  // Return any other character
```

```
%%
int yywrap(){return 1;}
```

Output:

```
C:\Users\comp\Desktop\SPCC>flex Exp3lex.l

C:\Users\comp\Desktop\SPCC>bison -d Exp3.y

C:\Users\comp\Desktop\SPCC>gcc lex.yy.c Exp3.tab.c -o parser

C:\Users\comp\Desktop\SPCC>parser
Enter expression:2*3+5/2-1+10
Result 17
C:\Users\comp\Desktop\SPCC>
```

Experiment No 4

2 Pass Assembler(Pass1)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void passOne(char label[10], char opcode[10], char operand[10], char code[10], char
mnemonic[3]);
void display();

int main()
{
    // for reading from input
    char label[10], opcode[10], operand[10];
    // for reading from optab
    char code[10], mnemonic[3];
    // call the function
    passOne(label, opcode, operand, code, mnemonic);

    return 0;
}

void passOne(char label[10], char opcode[10], char operand[10], char code[10], char
mnemonic[3])
{
    int locctr, start, length;

    FILE *fp1, *fp2, *fp3, *fp4, *fp5;                          // file pointers

    // read mode
    fp1 = fopen("input.txt", "r");
    fp2 = fopen("optab.txt", "r");
    // write mode
    fp3 = fopen("symtab.txt", "w");
    fp4 = fopen("intermediate.txt", "w");
    fp5 = fopen("length.txt", "w");

    fscanf(fp1, "%s\t%s\t%s", label, opcode, operand);          // read first line

    if (strcmp(opcode, "START") == 0) {
```

```
        // atoi() requires stdlib.h header file
        start = atoi(operand);                              // convert operand value from string to
integer and assign to start
        locctr = start;
        fprintf(fp4, "\t%s\t%s\t%s\n", label, opcode, operand);     // write to output file
(additional tab space as start will not have any locctr)
        fscanf(fp1, "%s\t%s\t%s", label, opcode, operand);          // read next line
    }
    else {
        locctr = 0;
    }

    // iterate till end
    while (strcmp(opcode, "END") != 0) {

        // 1. transfer address and read line to output file
        fprintf(fp4, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);

        // 2. make symtab file with values not starting with **
        if (strcmp(label, "**") != 0) {
            fprintf(fp3, "%s\t%d\n", label, locctr);
        }

        // 3. read from optab (code and mnemonic value)
        fscanf(fp2, "%s\t%s", code, mnemonic);

        // 4. traverse till the end of optab file
        while (strcmp(code, "END") != 0) {
            if (strcmp(opcode, code) == 0) {                    // if opcode in input matches the
one in optab, increment locctr by 3
                locctr += 3;
                break;
            }
            // read next line
            fscanf(fp2, "%s\t%s", code, mnemonic);
        }

        // 5. Searching opcode for WORD, RESW, BYTE, RESB keywords and updating
locctr
```

```
            // WORD -> add 3 to locctr
        if (strcmp(opcode, "WORD") == 0) {
            locctr += 3;
        }
            // RESW -> add 3*operand to locctr
        else if (strcmp(opcode, "RESW") == 0) {
            locctr += (3 * (atoi(operand)));                    // convert operand to integer and
multiply with 3
        }
            // BYTE -> add 1 to locctr
        else if (strcmp(opcode, "BYTE") == 0) {
            ++locctr;
        }
            // RESB -> add operand to locctr
        else if (strcmp(opcode, "RESB") == 0) {
            locctr += atoi(operand);
        }

        // read next line
        fscanf(fp1, "%s\t%s\t%s", label, opcode, operand);
    }
    // 6. transfer last line to file
    fprintf(fp4, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);

    // 7. Close all files
    fclose(fp4);
    fclose(fp3);
    fclose(fp2);
    fclose(fp1);

    // 8. display outputs
    display();

    // 9. calculate length of program
    length = locctr - start;
    fprintf(fp5, "%d", length);
    fclose(fp5);
    printf("\nThe length of the code : %d\n", length);
}
```

```
void display() {

    char str;
    FILE *fp1, *fp2, *fp3;

    // 1. Input Table
    printf("\nThe contents of Input Table :\n\n");
    fp1 = fopen("input.txt", "r");
    str = fgetc(fp1);
    while (str != EOF) {
        printf("%c", str);
        str = fgetc(fp1);
    }
    fclose(fp1);

    //2. Output Table
    printf("\n\nThe contents of Output Table :\n\n");
    fp2 = fopen("intermediate.txt", "r");
    str = fgetc(fp2);
    while (str != EOF) {
        printf("%c", str);
        str = fgetc(fp2);
    }
    fclose(fp2);

    // 3. Symtable
    printf("\n\nThe contents of Symbol Table :\n\n");
    fp3 = fopen("symtab.txt", "r");
    str = fgetc(fp3);
    while (str != EOF) {
        printf("%c", str);
        str = fgetc(fp3);
    }
    fclose(fp3);
}
```

—----------

Input.txt

```
**      START       2000
**      LDA    FIVE
```

```
**      STA   ALPHA
**      LDCH        CHARZ
**      STCH        C1
ALPHA       RESW        2
FIVE  WORD        5
CHARZ       BYTE  C'Z'
C1    RESB        1
**      END   **
```

Optab.txt

```
LDA 03
STA   0f
LDCH 53
STCH 57
END   *
```

Intermediate.txt

```
        **      START       2000
2000    **      LDA   FIVE
2003    **      STA   ALPHA
2006    **      LDCH CHARZ
2009    **      STCH C1
2012    ALPHA       RESW        2
2018    FIVE  WORD        5
2021    CHARZ       BYTE  C'Z'
2022    C1    RESB 1
2023    **      END   **
```

Symtab.txt

```
ALPHA       2012
FIVE  2018
CHARZ       2021
C1    2022
```

Length.txt
23

Experiment no 05

2 Pass Assembler(Pass2)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void display();

int main()
{
    char a[10], ad[10], label[10], opcode[10], operand[10], symbol[10];
    int start, diff, i, address, add, len, actual_len, finaddr, prevaddr, j = 0;
    char mnemonic[15][15] = {"LDA", "STA", "LDCH", "STCH"};
    char code[15][15] = {"33", "44", "53", "57"};

    FILE *fp1, *fp2, *fp3, *fp4;
    fp1 = fopen("output.txt", "w");
    fp2 = fopen("symtab.txt", "r");
    fp3 = fopen("intermediate.txt", "r");
    fp4 = fopen("objcode.txt", "w");

    fscanf(fp3, "%s\t%s\t%s", label, opcode, operand);

    while (strcmp(opcode, "END") != 0)
    {
```

```
    prevaddr = address;
    fscanf(fp3, "%d%s%s%s", &address, label, opcode, operand);
}
finaddr = address;

fclose(fp3);
fp3 = fopen("intermediate.txt", "r");

fscanf(fp3, "\t%s\t%s\t%s", label, opcode, operand);
if (strcmp(opcode, "START") == 0)
{
    fprintf(fp1, "\t%s\t%s\t%s\n", label, opcode, operand);
    fprintf(fp4, "H^%s^00%s^00%d\n", label, operand, finaddr);
    fscanf(fp3, "%d%s%s%s", &address, label, opcode, operand);
    start = address;
    diff = prevaddr - start;
    fprintf(fp4, "T^00%d^%d", address, diff);
}

while (strcmp(opcode, "END") != 0)
{
    if (strcmp(opcode, "BYTE") == 0)
    {
        fprintf(fp1, "%d\t%s\t%s\t%s\t", address, label, opcode, operand);
        len = strlen(operand);
        actual_len = len - 3;
        fprintf(fp4, "^");
        for (i = 2; i < (actual_len + 2); i++)
        {
            itoa(operand[i], ad, 16);
            fprintf(fp1, "%s", ad);
            fprintf(fp4, "%s", ad);
        }
        fprintf(fp1, "\n");
    }

    else if (strcmp(opcode, "WORD") == 0)
    {
        len = strlen(operand);
        itoa(atoi(operand), a, 10);
```

```
            fprintf(fp1, "%d\t%s\t%s\t%s\t00000%s\n", address, label, opcode, operand, a);
            fprintf(fp4, "^00000%s", a);
        }

        else if ((strcmp(opcode, "RESB") == 0) || (strcmp(opcode, "RESW") == 0)) {
            fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
        }

        else
        {
            while (strcmp(opcode, mnemonic[j]) != 0)
                j++;
            if (strcmp(operand, "COPY") == 0)
                fprintf(fp1, "%d\t%s\t%s\t%s\t%s0000\n", address, label, opcode, operand,
code[j]);
            else
            {
                rewind(fp2);
                fscanf(fp2, "%s%d", symbol, &add);
                while (strcmp(operand, symbol) != 0)
                    fscanf(fp2, "%s%d", symbol, &add);
                fprintf(fp1, "%d\t%s\t%s\t%s\t%s%d\n", address, label, opcode, operand,
code[j], add);
                fprintf(fp4, "^%s%d", code[j], add);
            }
        }

        fscanf(fp3, "%d%s%s%s", &address, label, opcode, operand);
    }

    fprintf(fp1, "%d\t%s\t%s\t%s\n", address, label, opcode, operand);
    fprintf(fp4, "\nE^00%d", start);

    fclose(fp4);
    fclose(fp3);
    fclose(fp2);
    fclose(fp1);

    display();
```

```
    return 0;
}

void display() {
    char ch;
    FILE *fp1, *fp2, *fp3, *fp4;

    printf("\nIntermediate file is converted into object code");

    printf("\n\nThe contents of Intermediate file:\n\n");
    fp3 = fopen("intermediate.txt", "r");
    ch = fgetc(fp3);
    while (ch != EOF)
    {
        printf("%c", ch);
        ch = fgetc(fp3);
    }
    fclose(fp3);

    printf("\n\nThe contents of Symbol Table :\n\n");
    fp2 = fopen("symtab.txt", "r");
    ch = fgetc(fp2);
    while (ch != EOF)
    {
        printf("%c", ch);
        ch = fgetc(fp2);
    }
    fclose(fp2);

    printf("\n\nThe contents of Output file :\n\n");
    fp1 = fopen("output.txt", "r");
    ch = fgetc(fp1);
    while (ch != EOF)
    {
        printf("%c", ch);
        ch = fgetc(fp1);
    }
    fclose(fp1);

    printf("\n\nThe contents of Object code file :\n\n");
```

```
    fp4 = fopen("objcode.txt", "r");
    ch = fgetc(fp4);
    while (ch != EOF)
    {
        printf("%c", ch);
        ch = fgetc(fp4);
    }
    fclose(fp4);

}
```

—----------

Output.txt

```
        **      START       2000
2000    **      LDA   FIVE  332018
2003    **      STA   ALPHA       442012
2006    **      LDCH  CHARZ       532021
2009    **      STCH  C1    572022
2012    ALPHA         RESW        2
2018    FIVE    WORD        5     000005
2021    CHARZ         BYTE  C'Z'  5a
2022    C1      RESB  1
2023    **      END   **
```

Objcode.txt

```
H^**^002000^002023
T^002000^22^332018^442012^532021^572022^000005^5a
E^002000
```

Experiment no 06

2 Pass Macro Processor(Pass 1)

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>
void main()
{
FILE *f1,*f2,*f3;
char mne[20],opnd[20],la[20];
//clrscr();
f1=fopen("minp2.txt","r");
f2=fopen("ntab2.txt","w+");
f3=fopen("dtab2.txt","w+");
fscanf(f1,"%s%s%s",la,mne,opnd);
while(strcmp(mne,"MEND")!=0)
{
if(strcmp(mne,"MACRO")==0)
{
fprintf(f2,"%s\n",la);
fprintf(f3,"%s\t%s\n",la,opnd);
}
else{
fprintf(f3,"%s\t%s\n",mne,opnd);}
fscanf(f1,"%s%s%s",la,mne,opnd);
}
fprintf(f3,"%s",mne);
fclose(f1);
fclose(f2);
fclose(f3);
printf("PASS 1 is successful");
//getch();
}
```

Minp2.txt

```
EX1     MACRO       &A,&B
-       LDA     &A
-       STA     &B
-       MEND -
SAMPLE      START       1000
-       EX1     N1,N2
N1      RESW 1
```

```
N2      RESW 1
-       END    -
```

ntab2.txt

```
EX1
```

dtab2.txt

```
EX1    &A,&B
LDA    &A
STA    &B
MEND
```

Experiment No 07
2 Pass Macro Processor(Pass 2)

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>
void main()
{
FILE *f1,*f2,*f3,*f4,*f5;
int i,len;
char mne[20],opnd[20],la[20],name[20],mne1[20],opnd1[20],arg[20];
//clrscr();
f1=fopen("minp2.txt","r");
f2=fopen("ntab2.txt","r");
f3=fopen("dtab2.txt","r");
f4=fopen("atab2.txt","w+");
f5=fopen("op2.txt","w");
fscanf(f1,"%s%s%s",la,mne,opnd);
while(strcmp(mne,"END")!=0)
{
if(strcmp(mne,"MACRO")==0)
{
fscanf(f1,"%s%s%s",la,mne,opnd);
```

```
while(strcmp(mne,"MEND")!=0)
fscanf(f1,"%s%s%s",la,mne,opnd);
}
else
{
fscanf(f2,"%s",name);
if(strcmp(mne,name)==0)
{
len=strlen(opnd);
for(i=0;i<len;i++)
{
if(opnd[i]!=',')
fprintf(f4,"%c",opnd[i]);
else
fprintf(f4,"\n");
}
fseek(f2,SEEK_SET,0);
fseek(f4,SEEK_SET,0);
fscanf(f3,"%s%s",mne1,opnd1);
fprintf(f5,".\t%s\t%s\n",mne1,opnd);
fscanf(f3,"%s%s",mne1,opnd1);
while(strcmp(mne1,"MEND")!=0)
{
if((opnd1[0]=='&'))
{
fscanf(f4,"%s",arg);
fprintf(f5,"-\t%s\t%s\n",mne1,arg);
}
else{
fprintf(f5,"-\t%s\t%s\n",mne1,opnd1);}
fscanf(f3,"%s%s",mne1,opnd1);
}
}
else{
fprintf(f5,"%s\t%s\t%s\n",la,mne,opnd);}
}
fscanf(f1,"%s%s%s",la,mne,opnd);
}
fprintf(f5,"%s\t%s\t%s\n",la,mne,opnd);
fclose(f1);
fclose(f2);
fclose(f3);
fclose(f4);
fclose(f5);
```

```
printf("pass2");
//getch();
}
```

atab2.txt

```
N1
N2
```

op2.txt

```
SAMPLE      START       1000
.       EX1     N1,N2
-       LDA     N1
-       STA     N2
N1      RESW 1
N2      RESW 1
-       END     -
```

Experiment No 08
Lexical analyser

Output:

```
C:\Users\comp\Desktop\SPCC>gcc lexer.c -o output

C:\Users\comp\Desktop\SPCC>output
Lexical Analysis:

Keyword: int
Identifier: a
Unknown character: =
Number: 5
Delimiter: ,
Identifier: b
Unknown character: =
Number: 1
Delimiter: ,
Identifier: c
Delimiter: ;
```

Experiment No 09
Parser

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

typedef enum { NUM, PLUS, MINUS, MUL, DIV, LPAREN, RPAREN, END } TokenType;
```

```
typedef struct {
    TokenType type;
    int value;  // used for NUM
} Token;

const char *input;  // input string for parsing
Token currentToken;

// Function declarations
Token getNextToken();
void eat(TokenType type);
int factor();
int term();
int expr();

// Lexical analyzer (returns next token from the input)
Token getNextToken() {
    while (*input != '\0' && isspace(*input)) {
        input++;  // Skip whitespace
    }

    Token token;
    if (*input == '\0') {
        token.type = END;
    } else if (*input == '+') {
        token.type = PLUS;
        input++;
    } else if (*input == '-') {
        token.type = MINUS;
        input++;
    } else if (*input == '*') {
        token.type = MUL;
        input++;
    } else if (*input == '/') {
        token.type = DIV;
        input++;
    } else if (*input == '(') {
        token.type = LPAREN;
        input++;
    } else if (*input == ')') {
        token.type = RPAREN;
        input++;
    } else if (isdigit(*input)) {
        token.type = NUM;
        token.value = 0;
        while (isdigit(*input)) {
            token.value = token.value * 10 + (*input - '0');
            input++;
        }
```

```
    } else {
        token.type = END;  // End of input or invalid character
    }
    return token;
}

// Function to consume a token of the expected type
void eat(TokenType type) {
    if (currentToken.type == type) {
        currentToken = getNextToken();
    } else {
        printf("Error: Unexpected token\n");
        exit(1);
    }
}

// Parser function for parsing factors (numbers and parentheses)
int factor() {
    int result = 0;
    if (currentToken.type == NUM) {
        result = currentToken.value;
        eat(NUM);
    } else if (currentToken.type == LPAREN) {
        eat(LPAREN);
        result = expr();  // Recursively parse the expression inside parentheses
        eat(RPAREN);
    } else {
        printf("Error: Unexpected token in factor\n");
        exit(1);
    }
    return result;
}

// Parser function for parsing terms (factor * factor or factor / factor)
int term() {
    int result = factor();
    while (currentToken.type == MUL || currentToken.type == DIV) {
        TokenType op = currentToken.type;
        if (op == MUL) {
            eat(MUL);
            result *= factor();
        } else if (op == DIV) {
            eat(DIV);
            result /= factor();
        }
    }
    return result;
}
```

```
// Parser function for parsing expressions (term + term or term - term)
int expr() {
    int result = term();
    while (currentToken.type == PLUS || currentToken.type == MINUS) {
        TokenType op = currentToken.type;
        if (op == PLUS) {
            eat(PLUS);
            result += term();
        } else if (op == MINUS) {
            eat(MINUS);
            result -= term();
        }
    }
    return result;
}

int main() {
    const char *expression = "3 + 5 * (10 - 4)";  // Example input
    input = expression;

    currentToken = getNextToken();  // Get the first token
    int result = expr();  // Parse the entire expression
    printf("Result: %d\n", result);
    return 0;
}
```

Output:

```
C:\Users\comp\Desktop\SPCC>gcc parser.c -o demo

C:\Users\comp\Desktop\SPCC>demo
Result: 33
```

Experiment No 10
Intermediate code Generator

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

typedef struct {
```

```
    int value;  // Value for numbers
    char op;    // Operator for token (+, -, *, /, etc.)
    char var;   // Variable name (e.g., a, b, t1, t2)
    int type;   // Token type: NUM, OPERATOR, or IDENTIFIER
} Token;

const char *input;  // Input string for parsing
Token currentToken;  // Current token
int tempCount = 1;  // To generate unique temporary variable names

#define NUM 1
#define OPERATOR 2
#define IDENTIFIER 3
#define END 4

void getNextToken();
void eat(int type);
int factor();
int term();
int expr();
void generateIntermediateCode(char *operation, char *operand1, char *operand2, char *result);
void initialize();

void getNextToken() {
    while (*input != '\0' && isspace(*input)) {
        input++;  // Skip whitespace
    }

    if (*input == '\0') {
        currentToken.type = END;
    } else if (*input == '+') {
        currentToken.type = OPERATOR;
        currentToken.op = '+';
        input++;
    } else if (*input == '-') {
        currentToken.type = OPERATOR;
        currentToken.op = '-';
        input++;
    } else if (*input == '*') {
        currentToken.type = OPERATOR;
        currentToken.op = '*';
        input++;
    } else if (*input == '/') {
        currentToken.type = OPERATOR;
        currentToken.op = '/';
        input++;
    } else if (*input == '(') {
        currentToken.type = OPERATOR;
        currentToken.op = '(';
```

```
        input++;
    } else if (*input == ')') {
        currentToken.type = OPERATOR;
        currentToken.op = ')';
        input++;
    } else if (isdigit(*input)) {
        currentToken.type = NUM;
        currentToken.value = 0;
        while (isdigit(*input)) {
            currentToken.value = currentToken.value * 10 + (*input - '0');
            input++;
        }
    } else if (isalpha(*input)) {
        currentToken.type = IDENTIFIER;
        currentToken.var = *input;
        input++;
    } else {
        currentToken.type = END;  // End of input or invalid character
    }
}

void eat(int type) {
    if (currentToken.type == type) {
        getNextToken();
    } else {
        printf("Error: Unexpected token\n");
        exit(1);
    }
}

void generateIntermediateCode(char *operation, char *operand1, char *operand2, char *result) {
    if (operation[0] != '\0') {
        printf("%s = %s %s %s\n", result, operand1, operation, operand2);
    } else {
        printf("%s = %s\n", result, operand1);
    }
}

void initialize() {
    tempCount = 1;  // Reset tempCount for every new expression evaluation
}

int factor() {
    int result = 0;
    char tempVar[10];

    if (currentToken.type == NUM) {
        result = currentToken.value;
        sprintf(tempVar, "%d", result);
```

```
        char tempResult[10];
        sprintf(tempResult, "t%d", tempCount++);  // Use a new temporary variable
        generateIntermediateCode("", tempVar, "", tempResult);  // No operation needed for numbers
        eat(NUM);
    } else if (currentToken.type == IDENTIFIER) {
        result = 0;  // We'll handle the identifiers as operands
        char tempVar[2];
        tempVar[0] = currentToken.var;
        tempVar[1] = '\0';
        char tempResult[10];
        sprintf(tempResult, "t%d", tempCount++);  // New temporary variable
        generateIntermediateCode("", tempVar, "", tempResult);  // Variable as intermediate code
        eat(IDENTIFIER);
    } else if (currentToken.type == OPERATOR && currentToken.op == '(') {
        eat(OPERATOR);  // eat '('
        result = expr();  // Recursively parse the expression inside parentheses
        eat(OPERATOR);  // eat ')'
    } else {
        printf("Error: Unexpected token in factor\n");
        exit(1);
    }

    return result;
}

int term() {
    int result = factor();
    char tempVar1[10], tempVar2[10], tempVar3[10];
    sprintf(tempVar1, "t%d", tempCount - 1); // Use the previous temporary variable

    while (currentToken.type == OPERATOR && (currentToken.op == '*' || currentToken.op == '/')) {
        char op = currentToken.op;
        eat(OPERATOR);
        int right = factor();
        sprintf(tempVar2, "t%d", tempCount - 1); // Use the previous temporary variable

        if (op == '*') {
            sprintf(tempVar3, "t%d", tempCount++);
            generateIntermediateCode("*", tempVar1, tempVar2, tempVar3);
        } else if (op == '/') {
            sprintf(tempVar3, "t%d", tempCount++);
            generateIntermediateCode("/", tempVar1, tempVar2, tempVar3);
        }
    }
    return result;
}

int expr() {
    int result = term();
```

```
    char tempVar1[10], tempVar2[10], tempVar3[10];
    sprintf(tempVar1, "t%d", tempCount - 1); // Use the previous temporary variable

    while (currentToken.type == OPERATOR && (currentToken.op == '+' || currentToken.op == '-')) {
        char op = currentToken.op;
        eat(OPERATOR);
        int right = term();
        sprintf(tempVar2, "t%d", tempCount - 1); // Use the previous temporary variable

        if (op == '+') {
            sprintf(tempVar3, "t%d", tempCount++);
            generateIntermediateCode("+", tempVar1, tempVar2, tempVar3);
        } else if (op == '-') {
            sprintf(tempVar3, "t%d", tempCount++);
            generateIntermediateCode("-", tempVar1, tempVar2, tempVar3);
        }
    }
    return result;
}

int main() {
    const char *expression = "a + b * c";  // Example input
    input = expression;

    initialize();
    getNextToken();  // Get the first token
    expr();  // Parse the entire expression
    return 0;
}
```

```
t1 = a
t2 = b
t3 = c
t4 = t2 * t3
t5 = t1 + t4
```